Grammar-Constrained Symbolic Regression for Systematic Alpha Discovery

A Methodology Based on Financial Time Series Stylized Facts

Working Paper — Concept & Methodology

Patryk Kozak

May 2026

Abstract

This paper presents a methodology for systematic discovery of quantitative trading signals ("alphas") using symbolic regression constrained by a grammar derived from empirical regularities of financial time series. Rather than allowing the symbolic regression engine to search an unconstrained space of mathematical expressions, we pre-compute a set of features grounded in fifteen well-documented stylized facts of asset returns, including volatility clustering, the leverage effect, fat tails, the Taylor effect, and time-varying market efficiency. These features serve as atomic building blocks—primitives—from which the symbolic regression algorithm (PySR) constructs candidate alpha expressions using only simple algebraic operators. The methodology proceeds in three stages: (1) a synthetic recovery test to validate the grammar's expressive power, (2) application to real equity data with temporal train/validation/test splits and a multi-criterion statistical filter, and (3) policy fusion and deterministic code generation via SymPy for deployment to backtesting infrastructure. We argue that this grammar-constrained approach offers three advantages over unconstrained symbolic regression: a dramatically reduced search space, built-in econometric interpretability, and structural protection against overfitting. The paper provides complete implementation specifications sufficient for independent replication.

Keywords: symbolic regression, alpha discovery, stylized facts, signal processing, PySR, quantitative finance, grammar-constrained search, feature engineering

1. Introduction

The search for profitable trading signals—commonly referred to as "alphas" in the quantitative finance industry—has traditionally relied on two approaches. The first is hypothesis-driven: a

researcher proposes a specific relationship (e.g., “stocks with low trailing returns tend to revert”), formalizes it as a mathematical expression, and tests it against historical data. The second is data-driven: machine learning models are trained to predict forward returns from a large feature set, producing predictions that are effective but opaque. Each approach carries a fundamental limitation. Hypothesis-driven research is constrained by the researcher’s imagination and domain knowledge; data-driven models sacrifice interpretability and are prone to overfitting in the low signal-to-noise regime characteristic of financial returns.

Symbolic regression occupies a middle ground. Unlike conventional regression, which assumes a functional form and estimates parameters, symbolic regression searches the space of functional forms itself. Given a set of input variables and operators, it evolves mathematical expressions that best fit the data, producing closed-form equations that are both predictive and interpretable. The PySR library (Cranmer, 2023), which interfaces with the Julia-based SymbolicRegression.jl backend, has demonstrated strong performance on scientific discovery tasks in physics and biology. Its application to financial alpha discovery, however, presents a unique challenge: the search space of possible expressions is vast, financial returns are noisy with low signal-to-noise ratios, and the risk of overfitting is acute.

Kakushadze (2015) addressed a related problem in “101 Formulaic Alphas,” presenting a catalog of trading signals expressed as compositions of simple operators (rank, delay, correlation, rolling mean) applied to price and volume data. These formulas were hand-crafted by domain experts. Our work takes Kakushadze’s catalog as a conceptual starting point but replaces manual formula construction with automated discovery via symbolic regression. Critically, rather than leaving the search unconstrained, we define a grammar—a structured set of primitives and composition rules—that encodes established empirical regularities of financial time series, known as stylized facts.

The central insight of this paper is that stylized facts can serve as an econometrically motivated prior for symbolic regression. By pre-computing features that encode these regularities—such as the Hilbert transform envelope for volatility clustering, the Hurst exponent for persistence detection, or spectral entropy for time-varying market efficiency—we provide the symbolic regression engine with building blocks that already carry economic meaning. The engine then searches only for algebraic combinations of these atoms, a dramatically smaller space than the full space of arbitrary mathematical expressions. Every candidate expression produced by this constrained search is interpretable in terms of known financial phenomena, which facilitates both economic validation and risk management.

This paper is organized as follows. Section 2 provides background on symbolic regression, stylized facts, and the signal processing methods we employ. Section 3 presents the complete methodology, proceeding chronologically through grammar design, feature computation, symbolic regression configuration, statistical filtering, and code generation. Section 4 discusses why this approach is well-suited to the alpha discovery problem and addresses its limitations. Section 5 concludes with directions for future work. Throughout, we provide sufficient implementation detail for independent replication.

2. Background

2.1 Symbolic Regression and PySR

Symbolic regression is a form of supervised learning in which the model is not a fixed parametric function but a symbolic expression drawn from a space defined by a set of variables and operators. The algorithm searches this combinatorial space—typically using genetic programming or evolutionary strategies—to find expressions that minimize a loss function while remaining parsimonious. PySR (Cranmer, 2023) implements this via a multi-population evolutionary algorithm with adaptive parsimony pressure, Pareto-optimal complexity-loss tradeoffs, and support for custom operators and constraints. The user specifies binary operators (e.g., +, −, ×, ÷), unary operators (e.g., abs, sign, neg), nesting constraints, and complexity penalties. PySR returns a Pareto front of expressions ranked by complexity and loss, from which the user selects candidates for further evaluation.

2.2 Stylized Facts of Financial Time Series

The term “stylized facts” refers to empirical regularities observed across a wide range of financial instruments, markets, and time periods. Unlike ad hoc observations, stylized facts are robust, well-documented, and have persisted for decades. The seminal catalog by Cont (2001) identifies over a dozen such regularities. We enumerate the fifteen facts that form the basis of our grammar, along with their original documentation and economic interpretation. Each fact motivates one or more signal processing primitives in our feature set.

The fifteen stylized facts, in the order we address them, are: (1) excess kurtosis / fat tails, first documented by Mandelbrot (1963) and Fama (1965); (2) volatility clustering, formalized as the ARCH effect by Engle (1982); (3) the leverage effect, identified by Black (1976); (4) the volume-volatility correlation, documented by Karpoff (1987); (5) the absence of linear autocorrelation in returns; (6) the slow, power-law decay of autocorrelation in absolute returns, documented by Ding,

Granger, and Engle (1993); (7) negative skewness of returns; (8) aggregational Gaussianity; (9) gain/loss asymmetry; (10) short-term mean reversion; (11) intermediate-term momentum, documented by Jegadeesh and Titman (1993); (12) lead-lag effects; (13) coarse-fine volatility asymmetry, documented by Müller et al. (1997); (14) the Taylor effect, documented by Taylor (1986); and (15) time-varying market efficiency, formalized as the Adaptive Markets Hypothesis by Lo (2004).

2.3 Signal Processing as Feature Engineering

Signal processing provides a natural toolkit for encoding stylized facts as computable features. Financial time series, viewed as discrete signals, can be analyzed using techniques developed for signal analysis: the Hilbert transform extracts instantaneous amplitude and phase, capturing cyclicality and envelope dynamics; the discrete wavelet transform decomposes a signal into frequency-specific components, enabling multi-scale volatility analysis; spectral entropy quantifies the concentration of power across frequencies, serving as a proxy for predictability and market efficiency; and autocorrelation-based measures directly quantify the persistence or mean-reversion character of returns at various horizons. These transformations produce bounded, numerically stable outputs that can be compared across assets via cross-sectional normalization—a prerequisite for cross-sectional alpha construction.

3. Methodology

We present the methodology in the chronological order of execution. Each subsection corresponds to a stage of the pipeline: grammar design, feature computation, symbolic regression, statistical filtering, and code generation. The reader can follow these stages sequentially to replicate the entire process.

3.1 Grammar Design: From Stylized Facts to Primitives

The grammar defines the space of expressions that PySR is permitted to explore. It consists of three elements: (a) the primitives—pre-computed features that serve as atomic variables, (b) the operators—algebraic operations PySR applies to combine primitives, and (c) the composition rules—constraints on how primitives and operators may be combined.

The design principle is that each primitive must correspond to a documented stylized fact and produce a bounded, numerically stable output suitable for cross-sectional comparison. We do not include arbitrary transformations; every primitive has a specific econometric motivation. This is

the key difference from standard feature engineering, where features are selected by predictive power alone. Here, features are selected by theoretical grounding, and predictive power is discovered by PySR's search over their combinations.

The pipeline's layered architecture enforces a strict separation of concerns. Layer 0 contains raw market data: returns, volume, high, low, and close prices. Layer 1 applies signal processing primitives to raw data, producing derived time series (e.g., Hilbert amplitude of returns over a 20-day window). Layer 2 normalizes Layer 1 outputs cross-sectionally using percentile rank or z-score, making them comparable across assets. Layer 3 is PySR's search space, restricted to the four basic algebraic operators (+, −, ×, ÷) and two unary operators (abs, neg). A critical composition rule prohibits nesting of signal processing primitives: hilbert(wavelet(...)) is forbidden, because such compositions lack econometric motivation and produce numerically unstable outputs. PySR operates exclusively in Layer 3, combining normalized atoms from Layer 2.

We now describe the primitives derived from each stylized fact, grouped by the phenomenon they encode. For each group, we provide the economic motivation, the mathematical definition, and the output bounds. All primitives operate on rolling windows, because financial time series are non-stationary and global statistics are meaningless. The standard window set is {5, 10, 20, 60} trading days, corresponding approximately to one week, two weeks, one month, and one quarter.

3.1.1 Fat Tails (Stylized Fact 1)

Asset returns exhibit excess kurtosis: extreme returns occur more frequently than predicted by a Gaussian distribution. We encode this with three primitives. Rolling kurtosis measures the fourth standardized moment over a window, with positive values indicating heavier tails than normal. The Hill estimator approximates the power-law tail index by fitting a Pareto distribution to the largest absolute observations within the window; lower values indicate fatter tails and higher extreme risk. The extreme ratio computes the fraction of observations exceeding two standard deviations, divided by the fraction expected under normality; values greater than one confirm fat-tailed behavior. These primitives allow PySR to discover expressions that condition on the current tail regime—for instance, reducing exposure when tails are unusually heavy.

3.1.2 Volatility Clustering (Stylized Fact 2)

Large price movements tend to be followed by large movements, and small by small. This is the ARCH effect (Engle, 1982), the most robust of all stylized facts. We encode it through several complementary primitives. The autocorrelation of absolute returns at various lags directly measures the strength of clustering. The ratio of short-term to long-term realized volatility captures whether the market is currently in a high- or low-volatility regime relative to its recent history.

EWMA (exponentially weighted moving average) volatility with different half-lives provides faster-reacting volatility estimates. The volatility of volatility—the standard deviation of rolling standard deviations—measures the intensity of clustering itself, capturing whether volatility is stable or erratic.

3.1.3 Leverage Effect (Stylized Fact 3)

Negative returns increase future volatility more than positive returns of equal magnitude (Black, 1976). We measure this asymmetry with three primitives. The downside-upside volatility ratio computes the standard deviation of negative returns divided by the standard deviation of positive returns; values greater than one indicate leverage effect. The rolling correlation between returns and changes in volatility should be negative when the leverage effect is present. The GJR asymmetry measure, inspired by the GJR-GARCH model, computes the ratio of the mean squared negative return to the mean squared positive return.

3.1.4 Volume-Volatility Correlation (Stylized Fact 4)

Trading volume tends to increase with absolute returns. This is consistent with models of information arrival, where new information triggers both price movements and increased trading activity. Our primitives include rolling correlation between absolute returns and volume, relative volume (current volume divided by its rolling average), volume surprise (current volume as a fraction of its rolling maximum), and volume-weighted volatility, which weights return variance by concurrent trading activity, providing a more informative measure of "true" volatility than equal-weighted alternatives.

3.1.5 Absence of Linear Autocorrelation (Stylized Fact 5)

Returns themselves exhibit near-zero autocorrelation, consistent with weak-form market efficiency. However, small deviations from zero are exploitable and time-varying. We measure rolling first-order autocorrelation of returns and the Lo-MacKinlay variance ratio. The variance ratio tests whether multi-period return variance scales linearly with the holding period (as it would under a random walk); values above one suggest momentum, below one suggest mean reversion. These primitives allow PySR to detect and exploit periods when returns deviate from random walk behavior.

3.1.6 Slow Decay of Autocorrelation in Absolute Returns (Stylized Fact 6)

Unlike returns themselves, absolute returns exhibit long-range dependence: their autocorrelation decays slowly, following a power law rather than an exponential (Ding, Granger, and Engle, 1993). We capture this with the Hurst exponent, estimated via the rescaled range (R/S) method on rolling

windows. Values above 0.5 indicate persistence (long memory), below 0.5 indicate anti-persistence. We compute the Hurst exponent both for raw returns (detecting momentum vs. mean-reversion regimes) and for absolute returns (measuring the strength of volatility persistence). We also compute the ACF decay ratio—the autocorrelation of absolute returns at a long lag divided by its value at a short lag—as a direct measure of decay speed.

3.1.7 Negative Skewness (Stylized Fact 7)

Equity returns are negatively skewed: large downward movements are more frequent than large upward movements. We encode this with rolling skewness and downside frequency, the ratio of returns below a negative threshold to returns above the corresponding positive threshold. Cross-sectional rank of skewness identifies assets with unusually asymmetric return distributions relative to peers.

3.1.8 Aggregational Gaussianity (Stylized Fact 8)

As the aggregation period increases (from daily to weekly to monthly), the distribution of returns converges toward Gaussian. We measure this convergence via the kurtosis ratio: excess kurtosis of daily returns divided by excess kurtosis of aggregated (e.g., five-day) returns. A high ratio indicates that non-Gaussianity is concentrated at high frequencies, consistent with microstructure effects or event-driven dynamics.

3.1.9 Gain/Loss Asymmetry (Stylized Fact 9)

Market drawdowns tend to be steep and rapid, while recoveries are gradual. We capture this with the up-down run ratio (average duration of consecutive positive-return days divided by average duration of consecutive negative-return days) and the conditional mean ratio (mean positive return divided by the absolute value of mean negative return). Values of the run ratio above one indicate that gains develop slowly while losses arrive abruptly.

3.1.10 Mean Reversion (Stylized Fact 10)

At short horizons (one to five days), returns in liquid instruments tend to revert. We encode this via four complementary primitives. The Hodrick-Prescott cycle component extracts the deviation from a smooth trend, providing a pure mean-reversion target. The Hilbert transform amplitude and phase capture the cyclical structure of returns: amplitude measures the strength of the current cycle, and phase identifies the position within it (trough, ascending, peak, or descending). The zero-crossing rate measures how frequently demeaned returns change sign; a high rate indicates choppy, mean-reverting behavior. Finally, the normalized deviation from the rolling mean (z-score of returns relative to a rolling window) measures how far current returns are from their local average.

3.1.11 Momentum (Stylized Fact 11)

At intermediate horizons (one to twelve months), assets that have performed well tend to continue performing well, and vice versa (Jegadeesh and Titman, 1993). Our primitives include cumulative returns over multiple windows (20, 60, 120, and 252 days), the Jegadeesh-Titman momentum measure (12-month cumulative return minus the most recent month, to avoid short-term reversal contamination), trend strength (rolling Sharpe ratio), and trend linearity (R-squared of a linear fit to cumulative returns, measuring how clean the trend is).

3.1.12 Lead-Lag Effects (Stylized Fact 12)

Large, liquid assets tend to react to new information faster than small, illiquid ones. We measure this with rolling beta to the equal-weighted market return and the Hou-Moskowitz delay measure, which quantifies how much additional explanatory power lagged market returns have beyond contemporaneous market returns. High delay indicates an asset that reacts slowly to market-wide information—a potential source of predictability.

3.1.13 Coarse-Fine Volatility Asymmetry (Stylized Fact 13)

Low-frequency (coarse) volatility predicts high-frequency (fine) volatility better than the reverse (Müller et al., 1997). We capture this asymmetry using wavelet decomposition, which provides a natural multi-scale analysis. We compute wavelet energy (the variance of detail coefficients) at three decomposition levels, corresponding to approximately 2-day, 4-day, and 8-day fluctuations, and the wavelet noise-to-signal ratio (energy at level 1 divided by energy at level 3). We also compute the spectral power ratio, the energy in low frequencies divided by energy in high frequencies, providing a complementary Fourier-based perspective.

3.1.14 Taylor Effect (Stylized Fact 14)

The autocorrelation of |r|^d is maximized at d approximately equal to 1, not d = 2 (Taylor, 1986). In other words, absolute returns exhibit stronger serial dependence than squared returns. We measure this with the Taylor ratio (autocorrelation of |r| divided by autocorrelation of r^2) and the optimal Taylor exponent (the value of d that maximizes autocorrelation, estimated by grid search). Deviation from d = 1 may indicate unusual microstructure dynamics.

3.1.15 Time-Varying Market Efficiency (Stylized Fact 15)

Market efficiency is not constant; it fluctuates over time in response to changing market conditions, as formalized by Lo's (2004) Adaptive Markets Hypothesis (AMH). We measure time-varying efficiency with spectral entropy (the Shannon entropy of the normalized power spectral density, where low values indicate concentrated spectral power and thus greater predictability),

permutation entropy (an ordinal-pattern-based complexity measure robust to outliers), and the R-squared of a rolling AR(1) model (the fraction of return variance explained by the simplest linear forecast).

3.2 Feature Computation and Panel Construction

With the grammar defined, we proceed to compute all primitives for a given dataset. The input is a panel of N assets observed over T trading days, with OHLCV (open, high, low, close, volume) data for each asset on each day. Log returns are computed as the first difference of log close prices. Volume is normalized per asset (z-scored over time) to make it comparable across instruments with different trading activity levels.

Each primitive function takes as input a (T × N) matrix and a window parameter, and returns a (T × N) matrix of the same shape. The first w rows of each output are NaN (not a number), where w is the rolling window length, because the primitive requires at least w observations. After all primitives are computed, each raw output is also passed through a cross-sectional rank transformation, producing a companion feature in the [0, 1] interval. This doubles the feature count but ensures that PySR has access to both absolute and relative versions of each primitive.

The resulting feature matrices are stacked into a panel format suitable for PySR: each row represents a single (date, asset) observation, with columns corresponding to the feature values at that point. The target variable y is the forward return—the log return from time t+1 (or t+h for longer horizons)—computed with strict temporal integrity to prevent lookahead bias. All rows containing any NaN or infinite values are dropped. For a typical panel of 30 assets over 1,500 trading days, this produces approximately 40,000 clean observations with 60–80 features, depending on whether wavelet primitives are included.

3.3 Symbolic Regression Configuration

PySR is configured to search for expressions that combine the pre-computed primitives using basic algebra. The configuration reflects the principle that the primitives have already done the heavy lifting of feature engineering; PySR's task is to find the right combination, not to discover new transformations.

The binary operators are restricted to {+, −, ×, ÷}, and the unary operators to {abs, neg}. Trigonometric and exponential functions are excluded because they lack econometric motivation when applied to already-transformed primitives and dramatically expand the search space without commensurate benefit. Nesting constraints prevent double division (which causes numerical

instability) and limit multiplication depth to two. Complexity penalties assign a cost of 2 to division and 1 to all other operators, biasing the search toward additive and multiplicative compositions.

The maximum expression size is set to 18 nodes, tighter than the default, because the primitives already encode substantial complexity; deep expression trees over pre-processed features are almost certainly overfit. Parsimony pressure is set to 0.012, substantially higher than the default, to favor simpler expressions. The loss function is L1 (mean absolute error) rather than L2 (mean squared error), because L1 is more robust to the fat tails and outliers characteristic of financial return distributions. The algorithm runs with 30 populations of 50 individuals each, for up to 100 iterations or 900 seconds, whichever comes first. Deterministic mode is enabled for reproducibility.

3.4 Validation Protocol

The validation protocol is designed to maximize protection against overfitting while retaining the ability to detect genuine signals. It proceeds in three stages, with strict temporal separation between each stage.

3.4.1 Temporal Split

The data is divided into three contiguous, non-overlapping periods: train (60%), validation (20%), and test (20%). The split is strictly temporal: the training period comes first chronologically, followed by validation, followed by test. There is no randomization, no shuffling, and no overlap. This mirrors the real-world constraint that a trading system must be developed on past data and deployed on future data. PySR sees only the training period. The validation period is used to filter candidates. The test period is not touched until the final evaluation, and each candidate is evaluated on it exactly once.

3.4.2 Statistical Filter

PySR produces not a single expression but a Pareto front of dozens of candidates, ranked by complexity and training loss. All candidates are evaluated on the validation set using four criteria. The Information Coefficient (IC), defined as the Spearman rank correlation between the alpha signal and forward returns, computed cross-sectionally for each date and averaged, must exceed 0.015. This is a deliberately low threshold; the goal is to retain candidates with even weak signal for further analysis. IC stability, the fraction of validation-period dates on which the daily IC is positive, must exceed 55%. This ensures that the signal is not driven by a single regime or a few extreme observations. The annualized Sharpe ratio of a quintile long-short portfolio (long the top

quintile of assets by alpha, short the bottom quintile) must exceed 0.3. And daily turnover must remain below 60%, ensuring that the signal is not simply capturing microstructure noise that would be destroyed by transaction costs.

Additionally, we impose a complexity ceiling of 20 nodes. This is not a statistical criterion but a structural one: expressions exceeding this complexity are almost impossible to interpret economically and are overwhelmingly likely to be overfit. Candidates that pass all five criteria proceed to out-of-sample evaluation on the test set.

3.4.3 Complexity-Performance Analysis

A key diagnostic of grammar quality is the relationship between expression complexity and out-of-sample performance. In a well-designed grammar, simpler expressions should generalize at least as well as complex ones on average, because the primitives already encode the relevant structure and additional complexity captures noise. We plot validation IC and Sharpe as a function of expression complexity for all candidates (not just those that pass the filter) to verify this relationship. If complex expressions consistently outperform simple ones, the grammar is likely under-specified—the primitives are not capturing enough structure, and PySR is compensating with depth. If the relationship is flat or negative (simpler = better out-of-sample), the grammar is well-calibrated.

3.5 Synthetic Recovery Test

Before applying the methodology to real data, we validate the grammar's expressive power with a controlled synthetic experiment. We generate a panel of 50 synthetic assets over 1,000 trading days, with returns drawn from a factor model (a common market factor plus idiosyncratic noise). We plant a known alpha signal—for instance, α = cs_rank(ts_mean(returns, 5)) − cs_rank(ts_mean(returns, 20)), a simple mean-reversion signal—and inject it into forward returns with a controlled signal-to-noise ratio. We then compute the full primitive set, run PySR, and evaluate whether the recovered expression is correlated with the true planted alpha.

The recovery test is not expected to reproduce the exact formula (equivalent expressions can differ syntactically), but the recovered expression's Spearman correlation with the true alpha should be high (>0.7 for moderate signal strength). If recovery fails, the grammar lacks the primitives needed to express the planted signal, or PySR's configuration is inappropriate. This test is run before touching real data and serves as a sanity check on the entire pipeline.

3.6 Deterministic Code Generation via SymPy

A distinguishing feature of this methodology is that every alpha expression discovered by PySR can be deterministically translated into deployable code. PySR natively exports expressions as SymPy symbolic objects. SymPy's code generation facilities then produce Python source code (via pycode), NumPy-backed callable functions (via lambdify), or C/Fortran code for performance-critical environments. The resulting code is a pure function with no stochastic elements, no model parameters, and no hidden state. It takes feature values as input and returns a scalar alpha value as output.

This determinism has important practical consequences. The generated code can be version-controlled, audited, unit-tested, and deployed to production backtesting and live trading systems without modification. There is no "model serialization" step, no framework dependency, and no risk of behavior changing between environments. The alpha is its code, and the code is a mathematical expression—the most transparent form a trading signal can take.

4. The Alpha-Policy Separation Principle

This section addresses a conceptual distinction that is central to the methodology and, in our view, underappreciated in the quantitative finance literature: the separation between the *alpha expression* (the signal) and the *policy function* (the execution rule). We argue that conflating these two components is a primary source of both overfitting and opacity in systematic trading systems, and that their explicit separation enables a more rigorous, more interpretable, and ultimately more profitable research process.

4.1 What Is an Alpha Expression?

An alpha expression is a mathematical function that takes as input a set of observable market features at time t and produces as output a scalar score for each asset in the universe. This score represents the expression's estimate of the asset's relative attractiveness—its expected forward return relative to other assets. Critically, the alpha expression says nothing about what to do with this estimate. It does not specify whether to go long or short, how much capital to allocate, when to enter or exit a position, or how long to hold it. It is a pure prediction: a ranking of assets by expected future performance.

In our framework, alpha expressions are the output of the grammar-constrained symbolic regression stage. They are closed-form mathematical formulas composed of stylized-fact primitives and basic algebraic operators. For example, an expression such as

csrank_hilbert_phase_20 × csrank_hurst_deviation_60 − csrank_spectral_entropy_20 produces a score for each asset on each day. This score has no inherent unit, no inherent scale, and no inherent action associated with it. It is, in the language of reinforcement learning, an observation—not a policy.

4.2 What Is a Policy Function?

A policy function takes the alpha score as input and produces a concrete trading action as output. The action includes all dimensions of execution: the direction of the position (long, short, or flat), the size of the position (what fraction of capital to allocate), the entry rule (under what conditions to initiate the position), the exit rule (under what conditions to close it), and the holding duration (how long to maintain the position before reassessment). The policy function transforms a prediction into a decision.

Consider a simple example. Given an alpha score α for each asset, the following are all distinct policies that could be applied to the same alpha: a sign policy that goes long when $\alpha > 0$ and short when $\alpha < 0$ with equal position sizes; a z-score proportional policy that sizes positions proportionally to the cross-sectional z-score of α, clipped at two standard deviations; a quintile long-short policy that goes long the top 20% of assets by α and short the bottom 20% with equal weight; a threshold policy that only trades when $|\alpha|$ exceeds a minimum threshold, remaining flat otherwise; a regime-conditional policy that applies the alpha only when a separate regime indicator (e.g., the Hurst exponent of market returns) indicates a favorable environment; and a Kelly-criterion policy that sizes positions according to the estimated edge and variance, targeting growth-rate maximization. Each of these policies would produce a different equity curve from the same alpha expression. The alpha determines *what* to trade; the policy determines *how* to trade it.

4.3 Why the Separation Matters

The separation of alpha and policy is not merely an organizational convenience. It has deep methodological consequences that affect every aspect of the research process, from overfitting protection to economic interpretability to production deployment.

4.3.1 Overfitting isolation

The most important reason to separate alpha from policy is that it isolates the sources of overfitting. An alpha expression can overfit to historical patterns in returns. A policy function can overfit to historical patterns in the equity curve—learning to size up before good periods and size down before bad ones. When alpha and policy are optimized jointly, these two forms of overfitting

interact and amplify each other, making it nearly impossible to determine whether the backtest performance is driven by genuine signal, by policy overfitting, or by their interaction. By separating the two stages and requiring the alpha to demonstrate predictive value under a trivial, parameter-free policy (the quintile long-short) before any policy optimization begins, we ensure that the alpha's contribution is real and independently verifiable. Only alpha expressions that exhibit statistically significant positive correlation with forward returns—measured by the Information Coefficient, the Sharpe ratio of a naive portfolio, and IC stability across rolling windows—are promoted to the policy optimization stage. This sequential gating prevents the policy from compensating for a weak or spurious signal.

4.3.2 Economic interpretability

An alpha expression that survives the statistical filter under a naive policy has a clear economic interpretation: it identifies assets that are more or less attractive based on a combination of stylized-fact features. This interpretation is independent of how the alpha is traded. Adding a policy does not change what the alpha predicts; it only changes how the prediction is acted upon. This separation means that the researcher can evaluate the economic plausibility of the alpha—does this combination of mean-reversion timing and volatility regime detection make sense?—without being distracted by the details of position sizing or exit rules. Conversely, the policy can be evaluated and optimized on its own merits: does this sizing rule maximize growth rate, minimize drawdown, or achieve the best risk-adjusted return?

4.3.3 Modularity and reuse

The separation creates modular components that can be independently maintained, tested, and reused. A single alpha expression can be deployed with different policies in different contexts—one policy for a low-risk allocation, another for a high-conviction book. A single well-tested policy can be applied to multiple alpha expressions. When an alpha decays, it can be replaced without modifying the policy infrastructure. When a better policy is developed, it can be deployed across all surviving alphas without re-running the symbolic regression stage.

4.4 The Policy Optimization Stage

Policy optimization is a second-stage process that operates exclusively on alpha expressions that have already demonstrated predictive value. The input to this stage is a validated alpha expression—one that has passed the statistical filter on the validation set and shown positive out-of-sample IC. The objective is to find a policy function that, when composed with this alpha, maximizes the growth rate of the equity curve.

The choice of growth rate as the objective—rather than cumulative return, Sharpe ratio, or win rate—is deliberate. Growth rate maximization, formalized by the Kelly criterion, produces the fastest compound growth over time and is the natural objective for a capital allocator with a long horizon. It automatically balances expected return against risk: a policy that sizes too aggressively will suffer catastrophic drawdowns that destroy compound growth, while a policy that sizes too conservatively will underperform. The equity curve produced by a growth-rate-optimal policy should be monotonically increasing in expectation, with drawdowns that are bounded and recoverable.

The policy function can be optimized through several approaches, which differ in their complexity and overfitting risk. The simplest approach is a parametric grid search over a small family of policies: for instance, searching over the threshold level in a threshold policy, or the clip value in a z-score proportional policy. This has few degrees of freedom and is relatively safe. A more ambitious approach uses symbolic regression itself to discover the policy function—treating the alpha score, volatility, and regime indicators as inputs and the position size as the target to optimize. This is powerful but requires careful cross-validation because it introduces a second layer of expression search. The most advanced approach, inspired by the transformer-based reinforcement learning paradigm, treats the policy as a learned mapping from the state (alpha score, current position, portfolio risk, market regime) to the action (position size and direction), optimized to maximize the growth rate of the resulting equity curve. In this framing, the output of the system is not a predicted return but a trading action—a position size and direction that, when executed over time, produces a monotonically increasing capital trajectory.

4.5 Alpha-Policy Fusion

The fusion of alpha and policy—the composition of a validated signal with an optimized execution rule—is the final stage of the pipeline before deployment. The fused object is a complete trading rule: it takes raw market data as input and produces concrete position sizes as output, with no human intervention required. Mathematically, if $\alpha(x)$ is the alpha expression and $\pi(\alpha, s)$ is the policy function (where s represents additional state such as current position and portfolio risk), then the fused trading rule is $f(x, s) = \pi(\alpha(x), s)$.

Fusion is permitted only when the alpha has independently demonstrated predictive value. This is a hard gate, not a soft recommendation. If an alpha expression has negative or zero IC on the validation set, no policy—no matter how sophisticated—should be applied to it. A sophisticated policy applied to a worthless signal will appear to "work" in backtesting by implicitly fitting to the equity curve's noise, producing results that will not replicate out of sample. The sequential

structure of the pipeline—alpha discovery, then statistical filtering, then policy optimization, then fusion—exists precisely to prevent this failure mode.

When multiple alpha expressions survive the filter, the fusion stage also encompasses alpha combination. Each alpha is z-scored cross-sectionally to make the scores comparable, and the z-scored alphas are combined—initially with equal weights, and optionally with weights optimized to maximize the composite signal's out-of-sample Sharpe or growth rate. Highly correlated alphas (Spearman correlation > 0.8) should not both receive full weight; the simpler or more stable of the two should be preferred. The combined alpha is then passed to the policy function. This hierarchical structure—individual alpha discovery, individual alpha validation, alpha combination, policy optimization, and finally fusion—ensures that each component is validated independently before being composed with others.

4.6 Connection to the Transformer Paradigm

The separation of alpha and policy described above is closely related to a paradigm emerging in deep learning for finance, in which a transformer or similar sequence model is trained not to predict returns but to output trading actions directly. In this paradigm, the model's output at each time step is a position size and direction that, when executed, contributes to a monotonically increasing equity curve. The model is optimized end-to-end on the growth rate of the resulting capital trajectory, not on prediction accuracy.

Our framework can be understood as a decomposed, interpretable version of this paradigm. Where the transformer learns both the signal and the policy simultaneously as entangled components of a single neural network, we separate them explicitly: the symbolic regression stage discovers the signal (the alpha expression), and the policy optimization stage learns the execution rule. The advantages of our decomposition are interpretability (we can inspect and validate each component independently), modularity (components can be replaced independently), and overfitting control (each stage has its own validation). The disadvantage is that the decomposition may forgo some performance that the end-to-end approach captures through joint optimization of signal and action. However, in the low signal-to-noise regime of financial markets, we believe the regularization benefits of decomposition outweigh the potential performance loss from separated optimization.

A natural extension of this work would combine the two approaches: use grammar-constrained symbolic regression to discover the alpha expression, then train a lightweight transformer or recurrent network as the policy function, optimizing it to maximize the growth rate of the equity curve conditional on the alpha's signal. This hybrid approach would retain the interpretability and

overfitting protection of the symbolic alpha while allowing the policy to capture complex, state-dependent execution patterns that a parametric policy might miss.

5. Discussion

5.1 Why Grammar Constraints Improve Alpha Discovery

The argument for grammar-constrained symbolic regression rests on three pillars: search space reduction, built-in interpretability, and structural overfitting protection.

Search space reduction is the most straightforward advantage. An unconstrained PySR instance with 10 raw features and operators {+, −, ×, ÷, sin, exp, log} at tree depth 8 faces billions of candidate expressions, the vast majority of which are mathematically meaningless in a financial context (e.g., sin(volume) ÷ exp(open)). Our grammar replaces this with approximately 60–80 pre-computed primitives and only 6 operators ({+, −, ×, ÷, abs, neg}) at tree depth 4–5, reducing the effective search space by orders of magnitude. This allows PySR to converge faster and explore the relevant subspace more thoroughly.

Built-in interpretability follows from the grammar design. Every primitive has a documented economic interpretation, so every expression PySR produces can be read as a composition of known phenomena. An expression such as csrank_hilbert_phase_20 × csrank_hurst_deviation_60 can be interpreted as: “buy assets that are at the bottom of their price cycle (Hilbert phase near −1) and currently in a mean-reverting regime (Hurst < 0.5).” This interpretability is not cosmetic; it enables the researcher to evaluate whether the economic mechanism is plausible before examining the backtest, providing a crucial check against overfitting.

Structural overfitting protection arises from the separation of layers. The primitives are deterministic transformations with no parameters fitted to the target variable. The window lengths (5, 10, 20, 60 days) are fixed by market convention, not optimized. The only parameters PySR optimizes are the structure and coefficients of the algebraic expression in Layer 3, a far smaller number of degrees of freedom than a standard machine learning model. Combined with the parsimony pressure, complexity ceiling, and multi-criterion statistical filter, this architecture makes it substantially more difficult for the system to find spurious patterns that do not generalize out of sample.

5.2 The Role of Stylized Facts as Priors

It is worth emphasizing that stylized facts are not hypotheses—they are empirical regularities confirmed across decades of data, multiple asset classes, and numerous independent studies. Volatility clustering has been documented since Mandelbrot (1963). The leverage effect has been observed in equities, indices, and credit markets. The Taylor effect has been confirmed across foreign exchange, equities, and commodities. By building primitives on these regularities, we are not "data mining" the current dataset; we are incorporating prior knowledge that generalizes across markets and time periods. This is analogous to using physical conservation laws as inductive bias in scientific symbolic regression—a practice that has been shown to dramatically improve both sample efficiency and generalization (Cranmer et al., 2020).

5.3 Limitations and Caveats

This methodology has several important limitations that the practitioner should be aware of. First, the grammar encodes only known regularities. If the data contains a genuinely novel phenomenon not captured by any of the fifteen stylized facts, the grammar will not express it. To mitigate this, we recommend including a small number of raw features (e.g., raw rolling returns and volatility at standard windows) as "escape hatches" that allow PySR to discover patterns outside the grammar's scope.

Second, the backtest framework described here does not account for transaction costs, market impact, or liquidity constraints. All reported Sharpe ratios and returns are gross of costs. In practice, any alpha must be evaluated net of realistic cost assumptions before deployment. The turnover filter provides partial protection, but it is not a substitute for a full cost model.

Third, the methodology assumes that stylized facts are stable over time. While these regularities have been remarkably persistent historically, there is no guarantee that they will persist indefinitely. Regime changes, regulatory shifts, and structural market evolution can all alter the statistical properties of returns. We recommend re-running the symbolic regression periodically (e.g., quarterly) to detect decay in alpha quality and discover new expressions adapted to the current regime.

Fourth, the computational cost of the primitive computation step is non-trivial. Several primitives (Hurst exponent, permutation entropy, wavelet decomposition) require $O(T \times N \times W)$ operations where W is the window length, and some involve nested loops. For large universes ($N > 500$ assets) or high-frequency data, vectorized or GPU-accelerated implementations may be necessary.

5.4 Hyperparameter Optimization with Optuna

The PySR configuration contains meta-parameters—parsimony, maximum expression size, population counts, complexity penalties—that control the search strategy but do not directly affect the candidate expressions' economic content. These meta-parameters can be optimized using a Bayesian hyperparameter optimization framework such as Optuna (Akiba et al., 2019). The objective function for Optuna is the best validation IC achieved by PySR under a given configuration, subject to a complexity ceiling. Because the meta-parameters control search behavior rather than model specification, this optimization does not introduce additional overfitting risk, provided the validation set remains untouched during the Optuna optimization and a held-out test set is reserved for final evaluation.

We advise against using Optuna to optimize parameters of the primitives themselves (e.g., window lengths) or the statistical filter thresholds (e.g., minimum IC), as these optimizations directly interact with the data and can degrade the structural overfitting protection that the methodology is designed to provide.

6. Conclusion and Future Work

We have presented a methodology for systematic alpha discovery that combines symbolic regression with a grammar grounded in the empirical regularities of financial time series. The approach proceeds through a well-defined sequence of stages—grammar design, feature computation, constrained symbolic search, statistical filtering, policy optimization, and deterministic code generation—each with clear responsibilities and validation criteria. The resulting alpha expressions are interpretable, auditable, and deployable without modification.

The methodology's primary contribution is the recognition that stylized facts can serve as a principled prior for symbolic regression in finance, analogous to the role of conservation laws in physics-informed machine learning. By restricting the search space to expressions built from econometrically motivated primitives, we achieve a substantial reduction in overfitting risk while retaining the flexibility to discover novel combinations that a human researcher might not have considered.

6.1 Extending the Grammar Beyond Financial Stylized Facts

The grammar presented in this paper was constructed from financial stylized facts—empirical regularities documented within the econometrics and quantitative finance literature. However, the framework itself is domain-agnostic. The only requirement for a valid primitive is that it takes a

time series as input, produces a bounded and numerically stable output, and admits a plausible interpretation as a feature relevant to asset price dynamics. This requirement does not restrict the source of the transformation to finance. Feature engineering methods from other quantitative disciplines can serve as primitives, provided they satisfy two conditions: the transformation must capture a structural property of the time series that is not already represented in the existing primitive set, and there must be a coherent economic argument for why that property would relate to forward returns. Below we describe several cross-domain extensions that meet these criteria.

6.1.1 Topological Data Analysis

Topological data analysis (TDA), and in particular persistent homology, extracts shape-based features from data that are invisible to traditional statistical methods. Applied to financial time series, TDA operates on sliding-window point clouds: a window of consecutive returns is embedded in a higher-dimensional space using Takens' delay embedding theorem, and persistent homology computes the topological features (connected components, loops, voids) of the resulting point cloud across multiple scales. The output is a persistence diagram summarizing which features appear and disappear as the scale parameter varies.

The financial relevance of TDA has been empirically validated. Gidea and Katz (2018) demonstrated that changes in the topology of return time series—specifically, the appearance of persistent loops in the point cloud—serve as early warning signals for market crashes, detecting structural instability before it manifests in volatility or drawdown measures. Concretely, the primitives derivable from TDA include persistence entropy (the Shannon entropy of the persistence diagram, measuring topological complexity), the sum of persistence lifetimes (total "duration" of topological features, high before crashes), and the Wasserstein distance between consecutive persistence diagrams (measuring how rapidly the topological structure is changing). These features capture regime transitions that are fundamentally geometric in nature—a market whose return dynamics form stable loops is behaving differently from one whose point cloud is diffuse—and this geometric perspective is complementary to the frequency-domain and autocorrelation-based primitives in our current grammar.

6.1.2 Information Theory

Information-theoretic measures quantify statistical dependencies without assuming linearity or any particular distributional form. While our current grammar includes entropy-based primitives (spectral entropy, permutation entropy), the information-theoretic toolkit extends further in ways that are directly relevant to financial markets.

Transfer entropy measures the directional flow of information from one time series to another: how much does knowing the past of series X reduce uncertainty about the future of series Y, beyond what the past of Y itself provides? Unlike Granger causality, which is restricted to linear relationships, transfer entropy captures nonlinear information flow. As a primitive, rolling transfer entropy from market or sector returns to individual asset returns quantifies how "information-connected" an asset is to the broader market—a feature directly related to the lead-lag effects documented as Stylized Fact 12, but generalized beyond the linear regime. Mutual information between returns and volume, computed on rolling windows, captures the full nonlinear dependence between price movements and trading activity, subsuming the linear correlation captured by our current volume-volatility primitives. Conditional entropy of returns given volume measures the residual unpredictability of returns after accounting for volume information—a more rigorous proxy for market efficiency than spectral entropy alone.

6.1.3 Dynamical Systems and Recurrence Analysis

Methods from dynamical systems theory treat a time series as the observable output of an underlying deterministic or stochastic system, and seek to characterize the system's properties from the observed data. The most directly applicable tool is recurrence quantification analysis (RQA), which constructs a recurrence plot—a binary matrix indicating when the system's state at time i is close to its state at time j—and extracts statistical measures from its structure.

The primitives derivable from RQA include the recurrence rate (the density of recurrence points, measuring how often the system revisits past states), determinism (the fraction of recurrence points forming diagonal lines, measuring how predictable the system's trajectory is), and laminarity (the fraction forming vertical lines, measuring how often the system gets "trapped" in a state). In financial terms, high determinism suggests that the market is following identifiable patterns and may be more predictable; high laminarity suggests regime stasis or low liquidity. These features characterize the dynamics of the system rather than its statistical distribution, offering a perspective distinct from both the frequency-domain and the autocorrelation-based primitives in our current grammar. The largest Lyapunov exponent, measuring sensitivity to initial conditions, provides a rolling estimate of the system's predictability horizon—high values indicate chaotic dynamics where prediction decays rapidly, low values indicate more regular behavior that may support longer holding periods.

6.1.4 Seismology: Regime Change Detection

Seismology has developed robust methods for detecting the onset of anomalous events in noisy continuous signals—a problem structurally identical to regime change detection in financial

markets. The STA/LTA (Short-Term Average / Long-Term Average) trigger, the standard algorithm for earthquake detection, computes the ratio of a signal's energy in a short recent window to its energy in a longer background window. When this ratio exceeds a threshold, an "event" is declared. Applied to absolute returns or realized volatility, STA/LTA provides a principled, well-tested regime change detector that is conceptually related to our volatility ratio primitive but benefits from decades of engineering refinement in seismological applications, including recursive variants and characteristic-function formulations that improve detection sensitivity and reduce false triggers.

6.1.5 A Principle of Disciplined Extension

It is tempting to enumerate further domains—fluid dynamics (turbulence models for market volatility), neuroscience (phase-amplitude coupling for multi-scale interaction), ecology (predator-prey dynamics as a metaphor for market competition)—but we caution against undisciplined expansion of the grammar. Each additional primitive widens the search space that PySR must explore. A primitive that does not capture genuine structure in the data functions as pure noise in the feature set, degrading the signal-to-noise ratio and increasing the risk of spurious discovery. The extensions described above—topological data analysis, information theory, recurrence quantification, and seismological detection—meet a high bar: each has been independently validated on financial data in peer-reviewed literature, produces bounded and interpretable outputs, and captures a structural property not already represented by existing primitives.

The principle we advocate is one of disciplined extension: expand the grammar only when a candidate primitive (a) has a rigorous mathematical definition, (b) produces numerically stable, bounded output suitable for cross-sectional comparison, (c) captures a structural property of the time series that is demonstrably distinct from existing primitives, and (d) admits a plausible economic interpretation for why that property would relate to forward returns. Primitives that are merely analogical—borrowed from another domain because the metaphor is appealing rather than because the mathematics is applicable—should be excluded. The grammar's power comes not from its breadth but from its precision: every atom should carry genuine information, and PySR's task is to find the combinations that matter, not to search through a cluttered space of loosely motivated features.

6.2 Further Directions

Beyond grammar extension, several additional directions for future work present themselves. The grammar could be extended to include cross-asset primitives (e.g., sector-relative momentum, pairwise correlation dynamics) that capture inter-instrument relationships currently absent from

the per-asset framework. The policy layer could incorporate regime detection, using the same stylized-fact primitives as conditioning variables for position sizing. A systematic comparison with deep learning approaches (e.g., LSTMs, transformers) on the same data and evaluation framework would quantify the interpretability-performance tradeoff. Walk-forward validation with expanding windows would provide a more realistic assessment of alpha decay and grammar stability over time. Finally, the hybrid approach described in Section 4.6—symbolic alpha discovery combined with a learned policy function optimized for growth rate—represents what we believe is the most promising path toward production-grade systematic trading systems that are simultaneously powerful, interpretable, and robust.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

Black, F. (1976). Studies of Stock Price Volatility Changes. Proceedings of the Business and Economic Statistics Section, American Statistical Association, 177–181.

Cont, R. (2001). Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues. Quantitative Finance, 1(2), 223–236.

Cranmer, M. (2023). Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. arXiv preprint arXiv:2305.01582.

Cranmer, M., Sanchez-Gonzalez, A., Battaglia, P., Xu, R., Cranmer, K., Spergel, D., & Ho, S. (2020). Discovering Symbolic Models from Deep Learning with Inductive Biases. NeurIPS 2020.

Ding, Z., Granger, C. W. J., & Engle, R. F. (1993). A Long Memory Property of Stock Market Returns and a New Model. Journal of Empirical Finance, 1(1), 83–106.

Engle, R. F. (1982). Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation. Econometrica, 50(4), 987–1007.

Fama, E. F. (1965). The Behavior of Stock-Market Prices. Journal of Business, 38(1), 34–105.

Gidea, M., & Katz, Y. (2018). Topological Data Analysis of Financial Time Series: Landscapes of Crashes. Physica A: Statistical Mechanics and its Applications, 491, 820–834.

Jegadeesh, N., & Titman, S. (1993). Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency. Journal of Finance, 48(1), 65–91.

Kakushadze, Z. (2015). 101 Formulaic Alphas. Wilmott Magazine, 2016(84), 72–81. Also available as SSRN Working Paper No. 2701346.

Karpoff, J. M. (1987). The Relation Between Price Changes and Trading Volume: A Survey. Journal of Financial and Quantitative Analysis, 22(1), 109–126.

Lo, A. W. (2004). The Adaptive Markets Hypothesis. Journal of Portfolio Management, 30(5), 15–29.

Mandelbrot, B. (1963). The Variation of Certain Speculative Prices. Journal of Business, 36(4), 394–419.

Müller, U. A., Dacorogna, M. M., Davé, R. D., Olsen, R. B., Pictet, O. V., & von Weizsäcker, J. E. (1997). Volatilities of Different Time Resolutions. Journal of Empirical Finance, 4(2–3), 213–239.

Taylor, S. J. (1986). Modelling Financial Time Series. John Wiley & Sons.